Jivitesh Singh(wd24972), Euan Chan XuanXi (ah24354), Talal Aljallal(pk23715)

Parallel Implementation

1 Functionality and Design

1.1 Functionality Implemented

Our parallel implementation of Conway's Game of Life in Go using an atomic-counter worker pool with mutex and conditional variables. Our ticker goroutine uses a mutex and conditional variable to signal our distributor functions ever 2 seconds, which in turn triggers the `AliveCellsCount` events. SDL Communication is also handled using a mutex-based queue. The keyboard controls let you pause ('p'), save ('s'), and quit ('q').

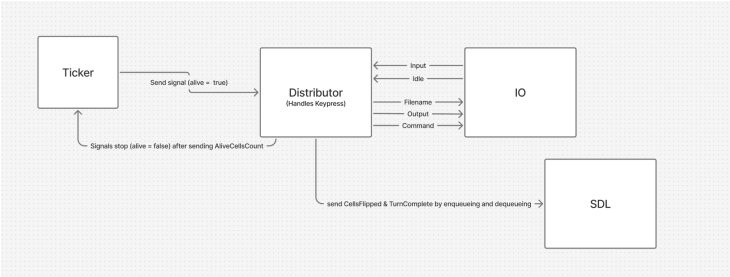

Figure 1: General Diagram on how the goroutines interact

1.2 Problems Solved

Note: This issue was solved during our previous implementation when we used channels to send a ticker signal to our distributor function. WaitGroup is no longer used in our distributor for our latest implementation

A problem that occured was that a warning appeared showing that the program had not returned from the `gol.Run` function. At first, we assumed the issue was that the main simulation loop was always checking for keypress input in the background, which took a lot of CPU power. As such, we used `time.Sleep(10 * time.Millisecond)` delay to solve this issue. However, we found out that the problem was that the `countSignal` goroutine was not included in the `WaitGroup`, which was causing potential goroutine leaks. Once this issue was resolved, we removed the time.Sleep, which drastically improved benchmarking performance by removing any unnecessary idle waiting between turns. (Benchmarking done in section 2.3).

2 Testing and Critical Analysis

Note: All benchmarking tests for parallel was done on the Acer Nitro 5 (11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz) unless stated otherwise.

2.1 If vs Mod vs Bitwise AND

Using the modulus operator to implement the wrap around has a significant performance cost in this implementation. The expression:

```
nx := (x + dx + width) % width
ny := (y + dy + height) % height
```

compiles to an integer division instruction. Integer division on modern CPUs is one of the higher latency arithmetic operations, taking lots of cycles. During the Game of Life, this execution occurs twice for each of the eight neighbours of every single cell. Over a 512 x 512 grid and 1000 iterations, this would be billions of division operations.

On the other hand, conditional statements wrap-around logic:

```
if nx < 0 {
    nx = width - 1
} else if nx >= width {
    nx = 0
} // horizontal wrap-around
```

consists of a smaller number of operations and assignments. The CPU learns to predict branch patterns, therefore, able to quickly learn them. This allows the CPU to execute with minimal overhead. Due to this, the branches with If statements executes with lower latency.

The expression `(x + dx + width) % width` is equivalent to:

```
(x + dx) & (width - 1)
```

This new version replaces the division with bitwise AND. Bitwise operations are one of the faster CPU instruction (typically completing in a single cycle and integrating well into the Pipeline.). The bitwise AND is a bit shift operation and it is known that shift operations are faster than modulus operations

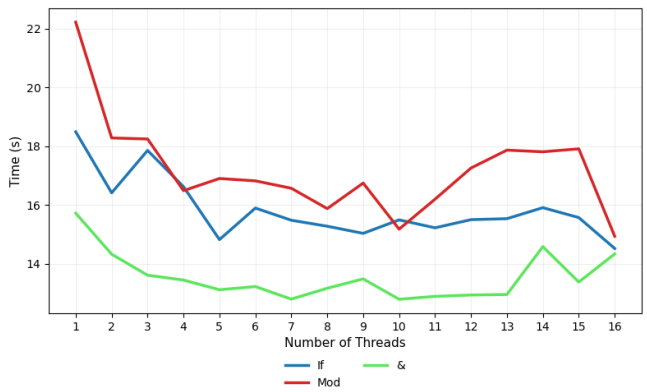

Figure 2: Performance between If, Mod and &

Adding on, the bitwise AND works in this case due to the fact that the grid dimensions are powers of two (e.g. 512x512). Taking an index modulo the width is mathematically equivalent to masking off the upper bits:

$$(x \bmod 2^n) \equiv x \& (2^n - 1)$$

The expression above states that when the divisor is a power of two, modulus can be replaced with a bitwise AND using a mask of one less than that power of two.

2.2 i7-11370H vs i5-3320M

Note: For this section, the benchmarking was done on the Acer Nitro 5 (11th Gen Intel(R) Core(TM) i7-11370H

@ 3.30GHz) and the Thinkpad T430 (Intel(R) Core(TM) i5-3320M CPU @ 2.60GHz) to compare performance between the architectures.

The code implementation used in this benchmark is based on the previously optimised version of the code, which incorporates the bitwise AND operation.

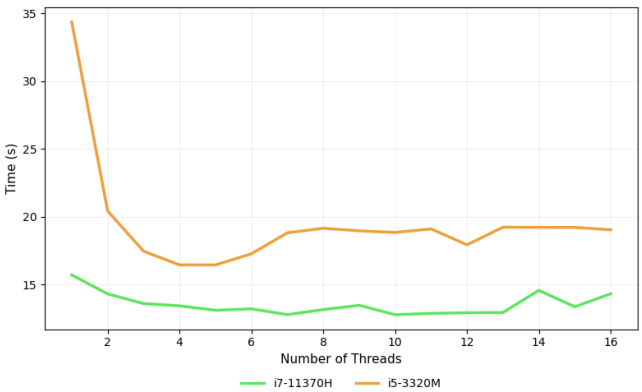

Figure 3: Performance between i7 and i5

Figure 3 compares the runtime performance of our Game of Life implementation on two CPUs, the Intel i7-11370H and the i5-3320M. Both show decreasing runtime with more threads. However, the i7 continuously outperforms the i5 across all the threads.

The difference in performance occurs due to architectural differences. The i7-11370H, built on Intel's Tiger Lake architecture, shows a higher instructions per cycle, wider execution piplines and a much faster integer arithmetic computations when compared to the Ivy-Bridge based i5-3320M.

Memory and cache would also affect the performance. The i7 features a larger 12MB L3 cache and a faster DDR4 memory, reducing the latency when repeatedly accessing the neighbouring cells. In contrast, the i5's smaller 3 MB cache and DDR3 bandwidth will limit the data reuse and could cause more cache misses. The i7 also benefits from improved prefetching, speculative execution, and branch prediction, which reduces idle cycles during grid traversal.

Lastly, the i7's four cores and eight threads provide greater parallel throughput than the i5's two cores and four threads. This can allow for smoother scaling up to around 8 threads before the performance plateaus. In contrast, the i5 saturates earlier due to thread contention and shared resource overheads.

2.3 The time.Sleep issue

Coming from the problems solved section, Figure 4 clearly shows the drastic improvement in our benchmarking after we removed the time.Sleep. Removing this allows our program to progress through the generations without any idle waiting. This significantly reduced overall execution time and improved throughput. (Figure 4)

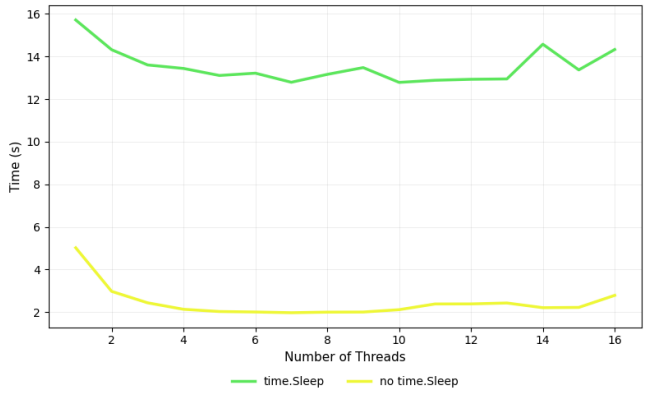

Figure 4: time.Sleep vs no time.Sleep

2.4 Memory Optimisation via Double Buffering

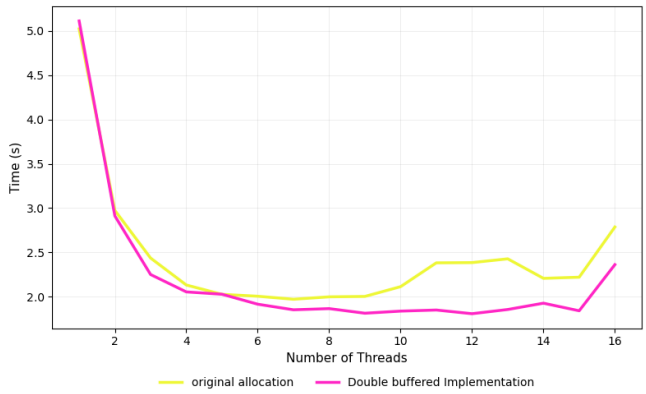

Figure 5: Performance after implementing double buffering

In our original nextStateParallel function created a new 2D world (newWorld) at the start of every generation:

```
newWorld := make([][]byte, height)
for i := range newWorld {
    newWorld[i] = make([]byte, width)
}
```

This means that for every turn, a new set of slices was allocated in memory. This in turn causes a notable allocation overhead, leading to an increased garbage collector (GC) pressure (needs to continuously recycle memory from discared slices), newly allocated slices might not occupy contigous memory which reduces spatial locality and increase cache misses and constant allocations and GC cycles could possibly interrupt CPU execution piplines

In my double buffered implementation, I allocated two equally sized 2D slices at the start of my distributor:

```
currentWorld := make([][]byte, p.ImageHeight)
nextWorld := make([][]byte, p.ImageHeight)
for i := 0; i < p.ImageHeight; i++ {
    currentWorld[i] = make([]byte, p.ImageWidth)
    nextWorld[i] = make([]byte, p.ImageWidth)
}
```

Every worker will write into `nextWorld` and `currentWorld` remains as a read-only for that generation. After a turn, instead

of allocating new memory, I swap their references which means that no new slices are created after initialisation and the simulation will run on a fixed, pre-allocated memory:

```
currentWorld, nextWorld = nextWorld, currentWorld
```

Images taken from mem.prof:

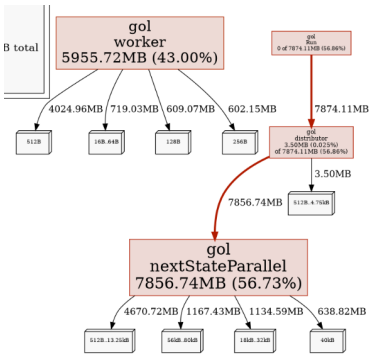

Figure 6: Before Memory optimisation

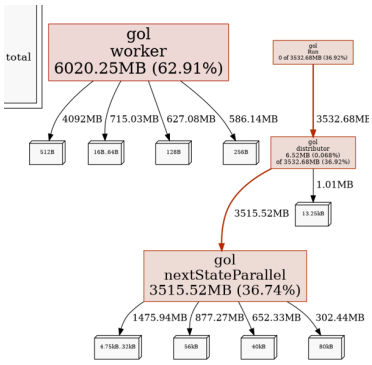

Figure 7: After Memory optimisation

2.5 Unrolling the Loop

From Figure 8, we can see that countAliveNeighbours function is dominating the CPU time (*Image taken from cpu.prof*):

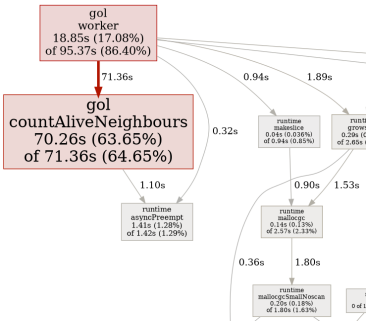

Figure 8: CPU performance before Unroll

The reason for this could be that two loops (dy and dx) are being executed, meaning, extra operations are happening for every single cell (such as increment operations), so across millions of cells, they add significant CPU overhead. We also learned that the CPU loves predictable patterns (as they are fast) and if it guesses it right, the performance goes up; if not, the performance tanks.

Therefore, we decided to unroll the loops for the eight neighbour checks

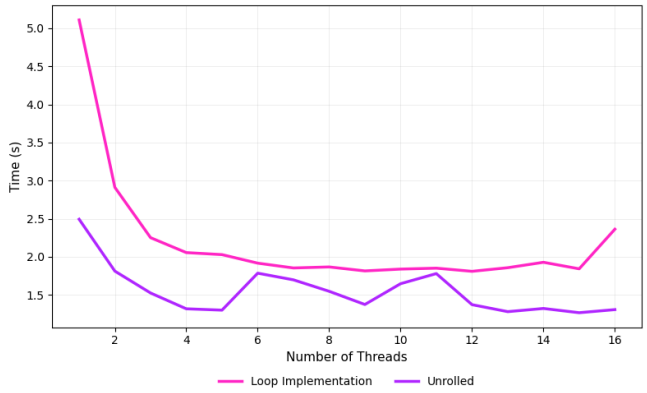

Figure 9: Loop vs No Loop

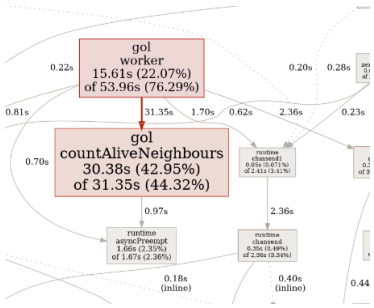

Figure 10: CPU Performance after unroll

2.6 Atomic Workers & Mutex with Conditional Variables for Ticker and SDL

2.6.1 Atomic Workers

In our previous implementation of workers, we used a channel based job queue that assigns each rows to workers through the jobs channel and they send back the computed rows through a results channel. This could cause contention on the channels. Each worker goroutine was assigned to a fixed contiguous block of rows to process. This works, but it can lead to poor load balancing. If one worker's row was cheaper to compute, that goroutine will finish early and sit idle while another worker was busy.

In our new implementation, workers write directly into shared memory, `nextWorld[y]` is for the next state and `flipsByRows[y]` is for the slice of flips for that row. All workers now share a single atomic counter `nextRow`. Every worker calls,

```
y := int(atomic.AddInt32(nextRow, 1) - 1)
```

(*"-1" as row indices in Go start from 0*)

to claim the next available row index before computing that row and no row will be skipped. The faster workers will process more rows while the slower ones will process fewer. This allows the work to be continuously redistributed among all threads.

2.6.2 Ticker using Mutex and Cond

In our original Implementation, the periodic counter for alive cells used a channel to send the signal and used a channel to stop

In our new implementation, the ticker uses a combination of a mutex, a condition variable, and two state flags (`alive` and

`stop`). Instead of sending a signal through a channel every two seconds, the ticker goroutine will set `alive = true` to indicate that a tick, then it will call `cond.Broadcast()` to wake up any goroutine interested in the event. When the distributor finishes it's work, it sets `stop = true` and broadcasts the condition variable, which allows the goroutine to exit clearly.

2.6.3 SDL using Mutex and Cond

In our original implementation, we updated SDL by sending `CellsFlipped` and then `TurnComplete` directly to `c.events` (SDL expects the events in that order). In our new implementation, the SDL system maintains a First - In - First - Out (FIFO) queue. When the SDL goroutine wakes up, it will dequeue the next event it receives from the distributer:

```
evt := s.queue[0]
s.queue = s.queue[1:]
```

The first line will read the front of the queue (the oldest event) and the second line will remove the event by re-slicing the queue to exclude index 0.

2.6.4 Benchmarking

A mutex protects the shared state while conditional variables will only wake up a goroutine that needs to run. However, in the case of channels, they would have to coordinate, which may cause overhead internally. (Figure 11)

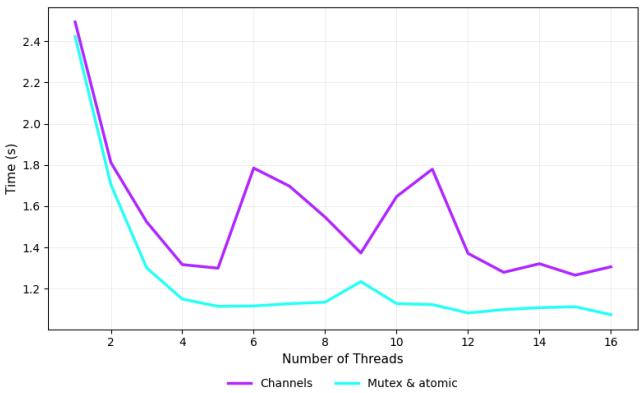

Figure 11: Performance after Implementing Mutex, Cond and atomic

2.7 Final Parallel Implementation

Note: For this section, the benchmarking was done on Acer nitro 5 (1th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz) and Lab Machine (Intel(R) Core(TM) i7-14700).

(Benchmarked using 64 threads)

The performance shown above reflects the major hardware differences between the Lab machine and the Acer Nitro 5. The Acer has 4 cores and 8 threads while the lab machine has 20 cores and 28 threads.

On the Acer, the performance improves quickly up to 8 threads but afterwards the threads provides no benefit. There is a spike observed at 49 threads, the number of workers do not fit nicely into 49 cores instead of 48 causing a 80,000ns spike in performance (context switching).

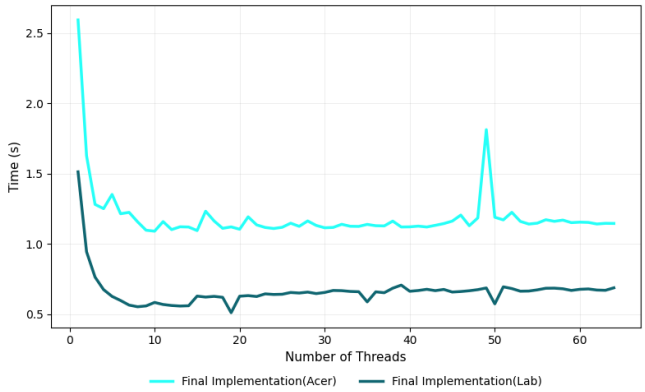

Figure 12: Final Parallel Implementation

On the lab machine, the performance continuous to scale further as it has more physical cores and threads (including a larger cache and a higher bandwidth memory). The small dips could be due to the introduction of a new thread.

Distributed Implementation

3 Functionality and Design

3.1 Functionality Implemented

Our distributed implementation introduces a broker-worker architecture that handles communication via RPC. The local controller (distributor.go) calls broker, passes the needed parameters for Game of Life logic, and calls for alive cells count every 2 seconds. It also calls broker for 'p', 'q', 's' and 'k' to pause, quit, save and shutdown the broker and workers.

The broker is designed to maintain the latest global Game of Life state, consisting current world and completed turns. Access to this shared state is protected by a mutex and a boolean, which ensure that concurrent key press triggered RPCs (such as pause, save, and shutdown) do not interfere with the main computation. For each turn, the broker partitions the board into four horizontal bands and, for each worker, sends the entire world together with the corresponding `start row` and `height` of that band to let each node compute in parallel.

Each worker node has three RPC operations: Firstly, computing next state of its assigned region in parallel (with parallel worker functions determined by the number of threads), Secondly, reporting the alive cells and number of it back to the broker, and finally shutting down cleanly when requested (presses 'k'). In this way, the broker orchestrates parallelism across multiple workers, while each worker exploits parallelism on its local CPU cores.

3.2 Network Communication

All communication between components is implemented using the go net/RPC package. A small set of request and response structs are define in stubs.go, which are transparently serialized by the RPC layer. This sub-section will outline what data is sent over the network, when it is sent, and why it is required.

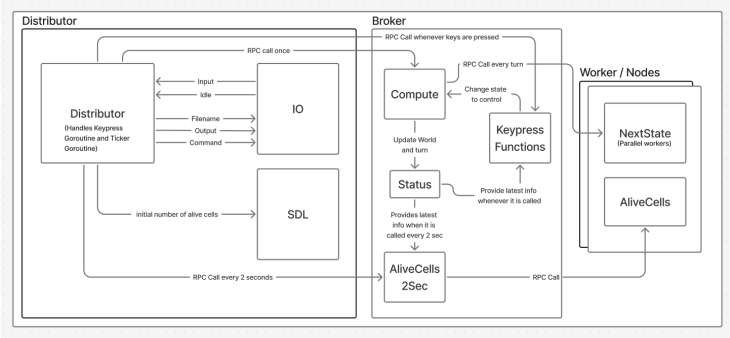

Figure 13: General Diagram on Components Interaction

Controller → Broker. At the beginning of the run, the controller sends the BrokerIn struct to the broker via the Broker.Compute RPC. The BrokerIn struct contains; the number of turns to simulate, the number of threads to be used inside each worker, the image height and width, and the initial world as a two-dimensional slice of bytes. This is done so that the broker can coordinate the evolution of the world without the controller being involved in the inner loop of the simulation.

During the execution, the controller will periodically send empty requests to Broker.AliveCells2Sec. These calls do not carry any world data; instead, they instruct the broker to return a small AliveCellCountOut struct containing the current completed turn and the number of alive cells. The controller then converts these into AliveCellsCount events for the SDL.

User input is also mapped to small RPC messages from the controller to the broker. The Keypress methods do not take arguments but return a Res struct which contains the latest world and turn. This allows the controller to generate PGM snapshots and to update the UI state in response to key presses.

Broker ↔ Workers. For each turn of the simulation the broker performs four RPC calls in parallel, one to each worker, and each call sends a WorkerIn struct the includes; the full current world, the image height and width, the per-worker horizontal band height, and the starting row index for that horizontal band. Sending the entire world to each worker is faster than only passing needed part as shown in 4.4.

Each worker responds with a Res struct that contains the updated horizontal band **World[start : start+height],** which the broker will receive for such partial worlds, then concatenates them in order of start row, and thereby reconstructs the full updated world for that turn. This updated world is the n stored in the broker's state and used as input for the next iteration.

At the end of the run the broker sends one more RPC to worker 0, passing a CellsIn struct with the final world so that the worker can compute the full list of alive cells. The resulting CellsOut is embedded into the BrokerOut returned to the controller.

Broker → Workers on shutdown. When the user presses k, the controller calls Broker.Shutdown. The broker will then set its internal quit and shutdown flags and returns the latest world to the controller for final output, and then sends an empty Shutdown request to each worker. These final RPC calls allow each worker to set its local quit flag and exit cleanly, avoiding resource leaks or partially completed work.

4 Testing and Critical Analysis

Note: In this section, benchmarking was done on (Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz) and (Intel(R) Core(TM) i7-14700) to compare performance between the architectures. Distributor and broker are executed locally and the 4 nodes are executed on different AWS instances by default.

4.1 Local Nodes vs AWS Nodes

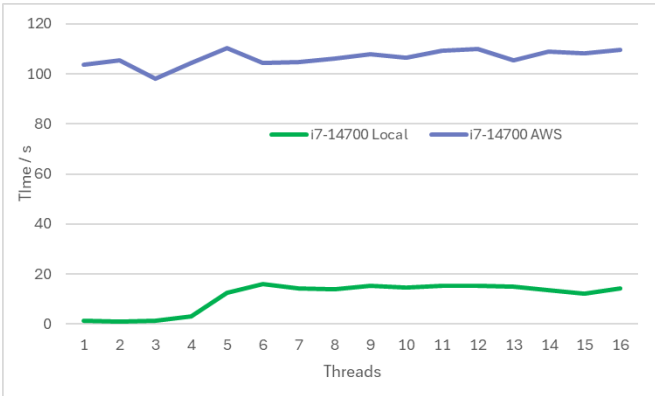

Figure 14: Local Nodes vs AWS Nodes

The time taken for the broker to communicate and pass data to each node to compute in each turn is directly linked to latency, which is affected by physical distance. The difference in time between local worker nodes and AWS worker nodes is due to the increased physical distance between the broker and each worker node.The broker contacts each node every turn and thus latency has impacted time geometrically, resulting in nearly fivefold time difference.

4.2 Local Broker vs AWS Broker

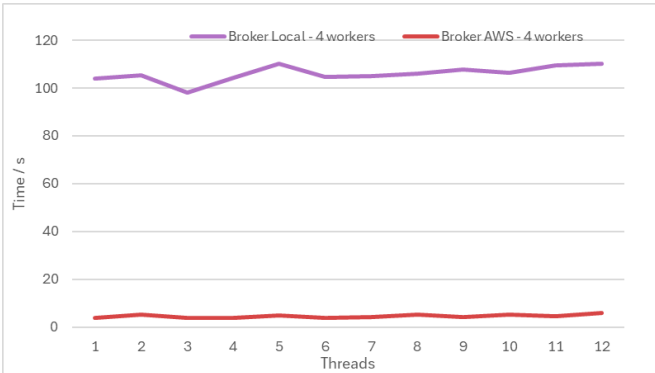

Figure 15: Local Broker vs AWS Broker

Initially we ran both the controller and broker locally, with all worker nodes running on separate AWS instances. Then we noticed that most RPC calls happen between the broker and the workers (not the controller and broker), so we decided to move the broker to AWS while keeping the controller local, expecting better performance. Figure 15 confirms our hypothesis: the broker on AWS is faster than the local broker because communication between AWS instances is quicker and more stable.

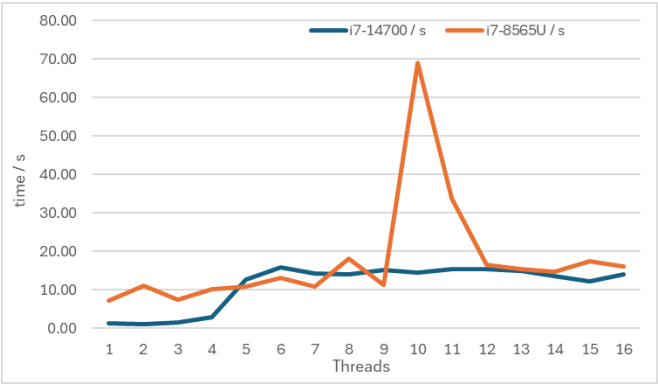

Figure 16: Difference Between Different Cores

4.3 i7-8565U vs i7-14700

The overall rise in runtime at higher thread counts is caused by overhead from coordinating the parallel workers in each node. Adding threads initially reduces the time to compute each turn, but beyond a certain point the extra scheduling and communication costs outweigh the benefits resulting in total time increase. The spike for the i7-8565U at 10–11 threads appears to be an outlier; we consider it unlikely to be due to thermal throttling or oversubscription and more likely due to network connection issues.

4.4 Passing Whole vs Passing Partial

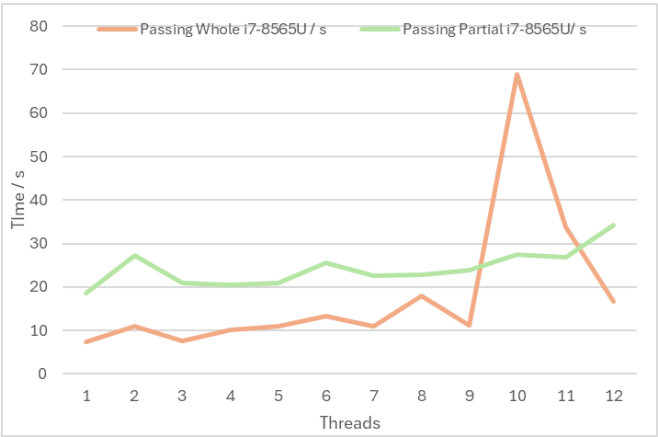

Figure 17: Passing Whole vs Passing Partial

The initial implementation was passing the whole world to each nodes every turn with specifications to let them compute specified part of world and pass the computed part back to broker. We thought of a possible optimization, which is to only pass the required part of world for nodes to compute next state, assuming that less data transferred between broker and each nodes would significantly speed of transferring data. However, the results shows that passing partial is generally taking more time than passing whole. The increased time by reallocating specified parts of world for every node in every turn has bigger magnitude than decreased time by fewer data transferred. It can be also due to sending data through RPC is still taking relatively long time and data is send in 1 package (request), hence causing relatively small decrease or even no decrease in transferring time.

4.5 Parallel Distributed workers

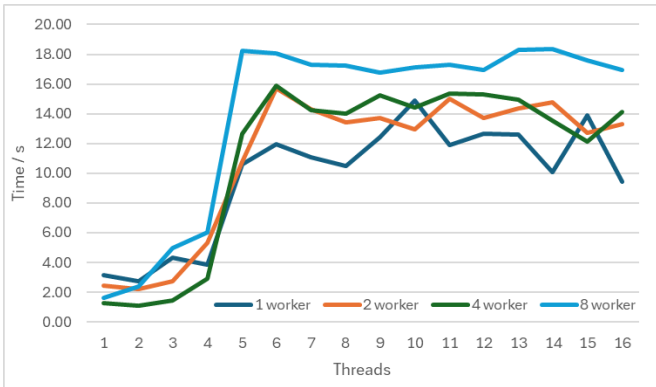

Figure 18: Different Numbers of Workers Nodes

Broker can attach more worker nodes but it might not be as efficient as the most optimized option of 4 workers nodes with 2 threads each, as shown in the graph. 1 worker node is generally faster than multiple nodes after 5 threads, which indicates increased time for distributing workload, from broker to nodes and from nodes to worker functions, are bigger in magnitude than decreased time for individual worker function compute time. The general trend of spiked and remained high after 5 threads might be due to realistic hardware limitations, that cores might be context switching, causing lower efficiency as there are threads allocated to each core more than what they can compute efficiently. It could also due to code limitation, such as not fully optimized worker function in each node causing longer runtime in every node.

4.6 Fault Tolerance

If the controller quits or is interrupted, it currently cannot reconnect to an existing broker/worker setup. In practice, this means that we have to restart the whole system, with the distributor beginning again from turn 0.

If the broker fails, the distributor will crash as well, while the workers remain running but unable to make progress until a new broker and distributor are started.

If the worker nodes fail, the broker will not be able to compute the next state correctly, since workers are only dialed at the start and assumed to be alive. This results in the broker blocking or sending incomplete results to the distributor, meaning restarting the entire system is needed.

5 Methodology

For performance evaluation, the parallel implementation's data was plotted using python, while the distributed was done so by Excel. Both versions were executed six times for each thread count to obtain a proper average. All our graphs our obtained by running the go benchmark tests.

We used cpu.prof and mem.prof to check on what functions to optimize in our parallel version, and in our distributed part it is used for checking for possible improvements.